02 Functions and built-in functions in Python

Introduction to Python - RSE course

Function? Why and what?

- *breaks* complex problems into smaller parts
- enables *re-use*
- makes code more *readable*

```
def average(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)
```

→ DEFINING A FUNCTION

- Begin the definition of a new function def.
- Followed by the name of the function.
- Then *parameters* in parentheses.
    - Empty parentheses if the function doesn't take any inputs.
- Then a colon and then an indented block of code
- Use return ... to give a value back to the caller
- The function doesn't need a return

Calling a function

- **function_name** – the name of the function
- **arguments** – values we pass to the function

```
def average(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)

a = average([1, 3, 4])
print('average of actual values:', a)
```

- **Defining** a function **does not run it.**
  - Like assigning a value to a variable.
- Must call the function to execute the code it contains

Variable scope

Local variables

- defined *inside a function*
- exist *only inside that function*

Global variables

- defined *outside functions*
- can be accessed *throughout the program*

```
initial_fine = 0.25
late_fine = 0.50

def calc_fine(days_overdue):
    if days_overdue <= 10:
        days_overdue =  days_overdue * initial_fine
    else:
        days_overdue = ((days_overdue * initial_fine) +
                        (days_overdue * late_fine))
    return days_overdue
```

Built-in Functions

- Python already provides many **built-in functions**.
- These are functions that are **available by default** — we can use them without importing any library.
- For example:
    a. `print()` → displays output
    b. `len()` → returns the length of an object
    c. `sum()` → adds numbers in a list
    d. `max()` / `min()` → finds largest or smallest value
    e. `sorted()` → sorts items in list
    f. `help()` → provides help and documentation for functions

Built-in Functions in Libraries

- Python already contains many useful **built-in functions**.
- But even more functionality is available in **libraries**.
- Libraries contain implementations of many **well-known algorithms and tools**, so you usually don’t need to implement them yourself.

```
→ NEXT WEEK WE WILL LOOK AT LIBRARY NUMPY
```

Key takeaways

- functions **group instructions into reusable blocks.**
- pass **arguments** to a function and receive a **result** using return.
- variables defined inside a function are **local** and exist only within that function.
- functions like print(), len(), sum() help us **write simpler and shorter code.**